Machine Learning and Deep Learning
Lecture-08

By: Somnath Mazumdar
Assistant Professor
sma.digi@cbs.dk

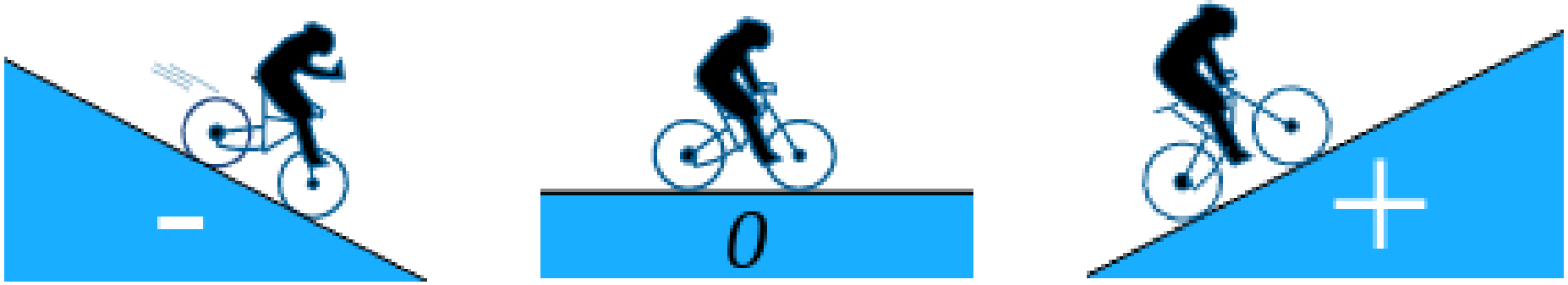
Spring 2026

Overview

- Gradient Descent
- Activation functions
- Backpropagation
- Optimal configurations:
- Grid Search/Random Search
- Adam/AdamW optimizer
- Early Stopping

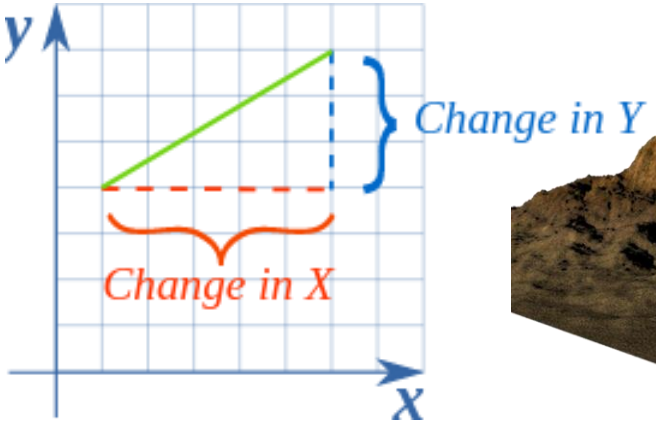
Gradient Descent

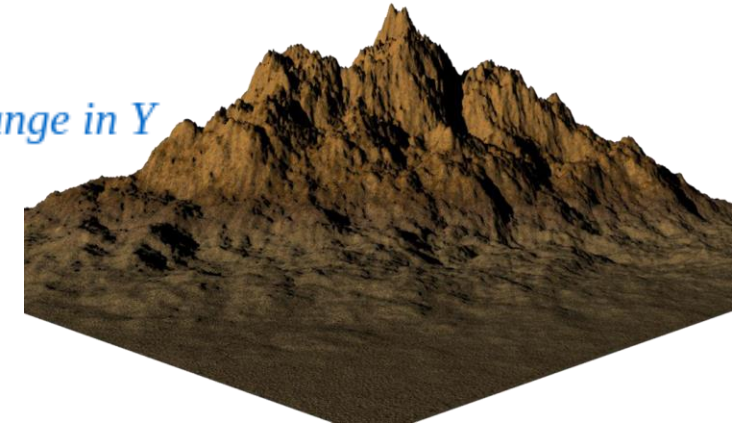
Gradient + Descent



Gradient presents slope
Descent: Moving down

- Gradient = Change in Y/ Change in X
- Gradient gives direction of steepest ascent
- Negative gradient → steepest descent



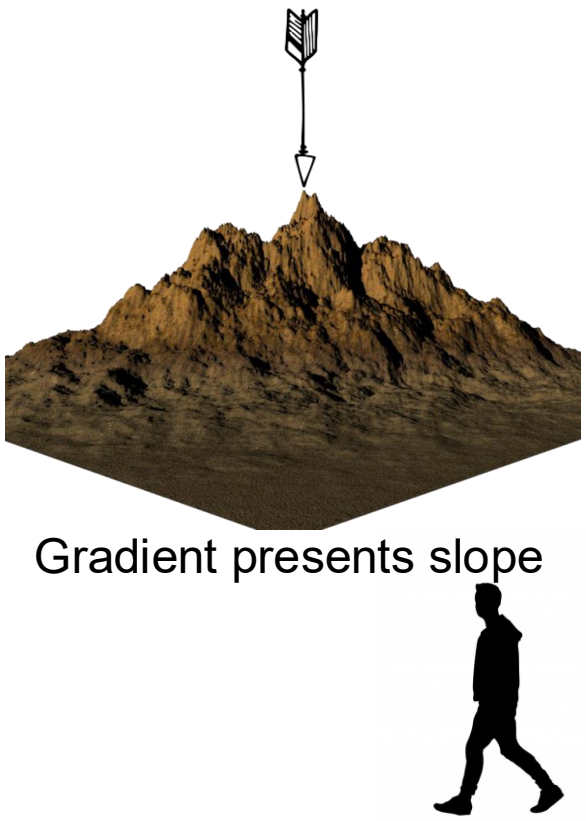


Gradient Descent (GD)

- An optimization algorithm
  - Goal: Minimize objective function
- Capable of finding optimal solutions to a wide range of problems.
- Stop when convergence criteria satisfied

- Normal equations only solve linear least squares problems.
  - Normal equations cost $O(nd^2 + d^3)$.

- GD solves many other problems.
  - GD costs $O(ndt)$ to run for 't' iterations.
    - GD can be faster when 'd' is very large.
- Stopping Criteria
  - Maximum iterations reached
  - Validation loss stops improving

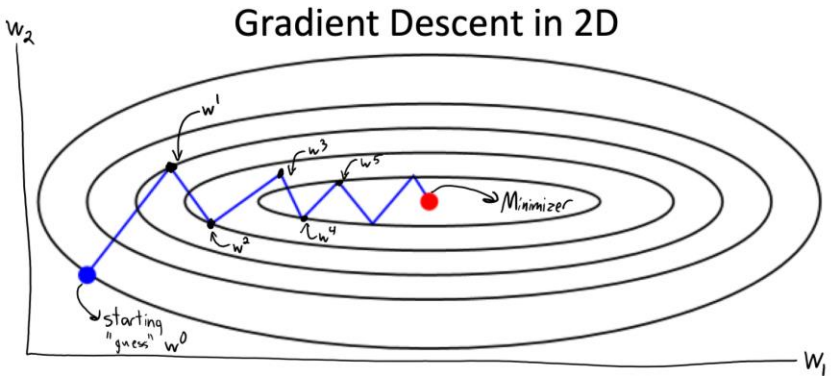


Gradient Descent (GD)

https://www.popsci.com/

Gradient Descent (GD) Definition

- Iterative parameter updates starting from w_0
- Use gradient $\nabla f(w_t)$ to generate better guesses
- Update magnitude is proportional to gradient norm, but step size is controlled by α (alpha).



$$w_{t+1} = w_t - \alpha_t \nabla f(w_t)$$

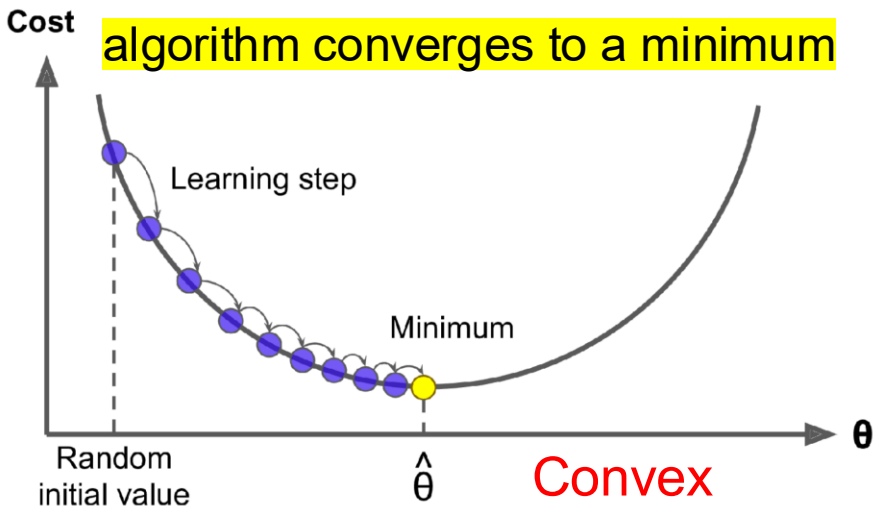
vector operator ∇ called del/nabla

- f differentiable
- ∇f Lipschitz continuous
- α_t properly chosen
- If f convex → convergence to global minimum
- If non-convex → convergence to stationary point

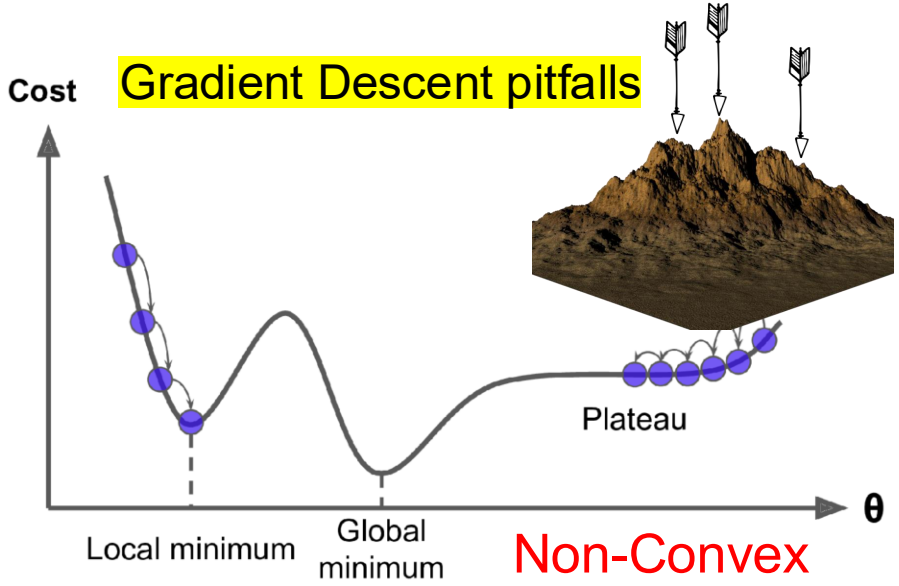
Gradient Descent (GD)

- Model parameters are initialized randomly.
- Tweak repeatedly to minimize cost function/MSE.
- Learning step size is proportional to slope of cost function
  - Steps gradually get smaller as parameters approach minimum.

- Random initialization starts algorithm on left.
- Converges to local minimum (global) minimum.
- Starts on right, will take a very long time to cross plateau.
  - If stops early, never reach global minimum.

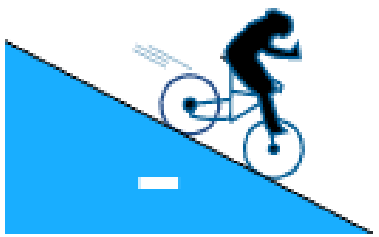


- Convergence Conditions
  - Convex f → convergence to global minimum
  - Strongly convex f → linear convergence rate
  - Non-convex f → convergence to stationary point



Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow by Aurélien Géron, 2019.

GD: Local Minimum

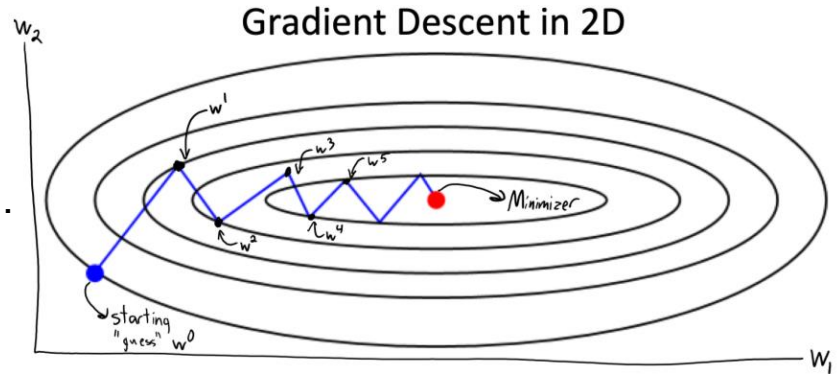


- Start with w^0 (initial guess)
- Generate new guess by moving in negative gradient direction:
  - It decreases 'f' if "step size" α^0 is small enough.
  - We decrease α^0 if it increases 'f'. **α = learning rate**

- Repeat (to successively refine guess): $w^{t+1} = w^t - \alpha^t \nabla f(w^t)$ for $t = 1, 2, 3, \ldots$

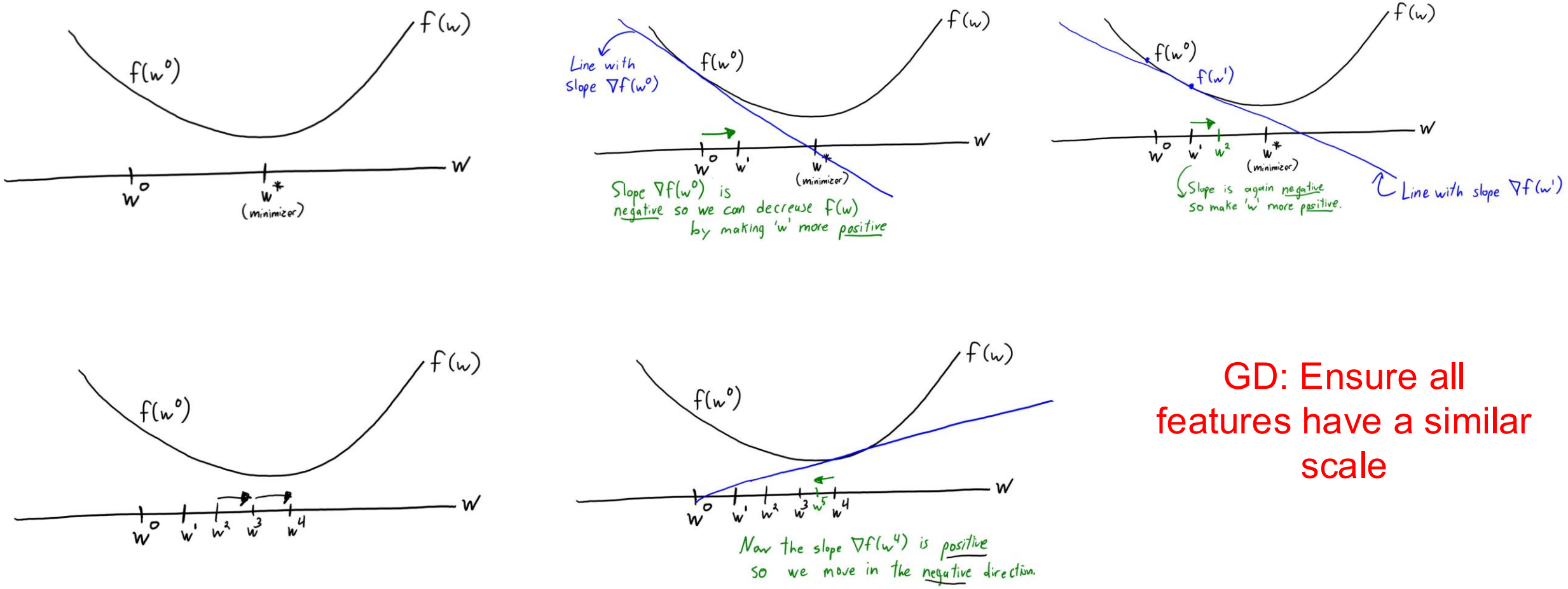
- Stop if not making progress

- Under weak conditions, algorithm converges to a 'w' with $\nabla f(w) = 0$.
- 'f' is bounded, ∇f doesn't change arbitrarily fast, small and constant α^t .



GD: Local Minimum

- GD based on: Given parameters ‘w’, direction of largest decrease is − ∇ f(w).



GD: Ensure all features have a similar scale

Learning Rate (α)

- Learning rate is a positive hyperparameter ($\alpha > 0$).
  - It scales the gradient in the update rule: $w_{t+1} = w_t - \alpha \nabla f(w_t)$
  - Controls magnitude of parameter updates.
  - Determines convergence speed.
- Effects of Learning Rate
  - Small $\alpha \rightarrow$ Stable and slow convergence (many iterations required)
  - Large $\alpha \rightarrow$ faster movement but risk of:
    - Overshooting
    - Oscillation
    - Divergence

Learning Rate/Step size

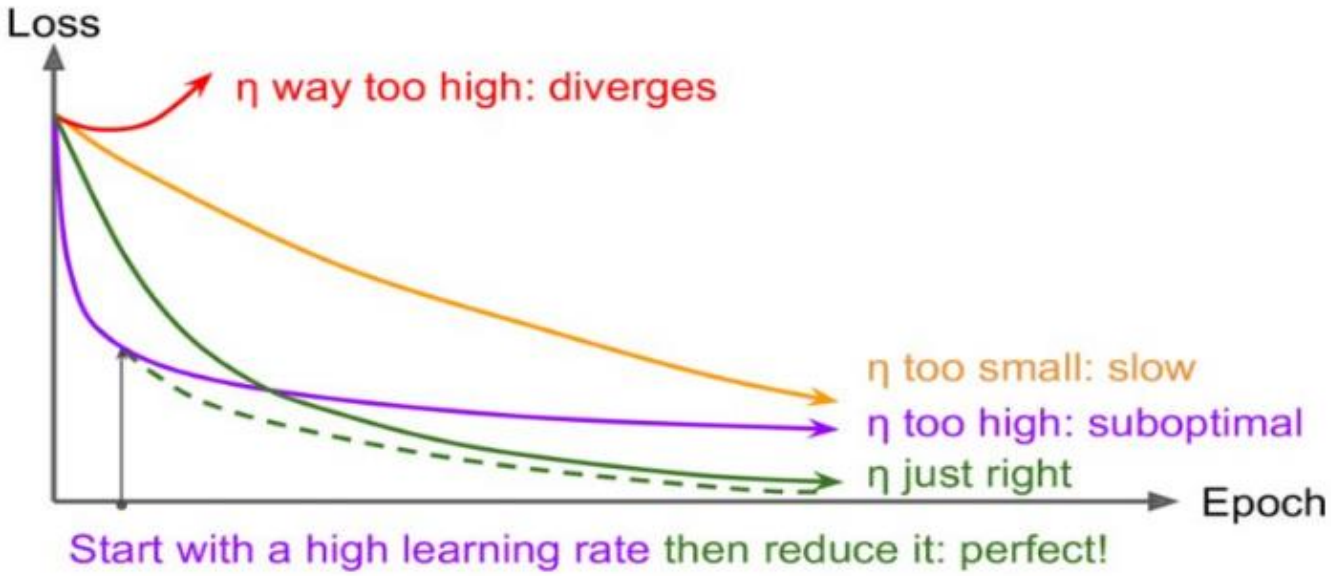
- **Deterministic GD**: A carefully chosen constant step size ensures convergence for smooth convex functions.
- **Stochastic GD**: Decreasing step sizes are typically needed for convergence.
- Decaying too quickly may cause premature stagnation.
- In practice, constant or slowly decreasing step sizes often perform better.

Learning Rate: Setting the Step-Size

- SGD is sensitive to the learning rate, especially in deep neural networks.
- **Strategy**:
  - Start with a fixed learning rate.
  - Monitor training and validation loss.
  - If progress stalls or loss increases: Reduce the learning rate (learning rate decay).
- In some models, bias parameters may use slightly larger learning rates.
- However, modern optimizers (e.g., Adam) adapt learning rates automatically per parameter.
- SGD keeps a single learning rate (alpha) for all weight updates
  - Learning rate does not change during training.

Learning Rate Scheduling

- Steps to finding a good learning rate:
  - Initialize the model.
  - Train for a few hundred mini-batch iterations.
  - Gradually increase the learning rate exponentially from a very small value to a large value.
  - Record loss as a function of learning rate.
  - Identify the point where loss starts increasing sharply.
  - Choose a learning rate slightly below that value.
  - Reinitialize the model and train using the selected learning rate.



Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow by Aurélien Géron, 2019

Stochastic
Gradient Descent

Stochastic Gradient Descent (SGD)

- DNN are trained using SGD.
  - Estimates error gradient for current state from training data.
  - Updates weights of model using backpropagation.
- Approach to discriminative learning of linear classifiers under convex loss functions (such as (linear) SVM and Logistic Regression).
- Application: text classification, NLP.
- Pros: Efficiency, Ease of implementation
- Cons: Need hyperparameters (regularization parameter, no of iterations)
  - Sensitive to feature scaling.

Stochastic Gradient (SG)

- Minimize an empirical risk (average loss): $f(w) = \frac{1}{n}\sum_{i=1}^{n} f_i(w)$
- Algorithm:
  - Initialize parameters w^0.
  - Generate new guess by moving in –ve gradient direction $w^1 = w^0 - \alpha^0 \nabla f_i(w^0)$
  - Sample a random training example $i_t \in \{1,\ldots,n\}$
  - Update using stochastic gradient:

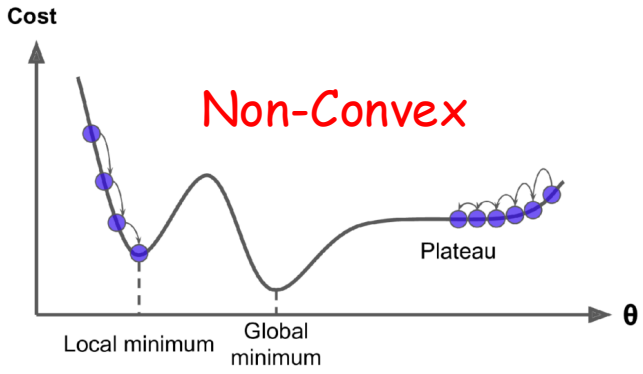
$$w^{t+1} = w^t - \alpha^t \nabla f_i(w^t) \quad \text{for } t = 1, 2, 3, \ldots$$

gradient from
a single example

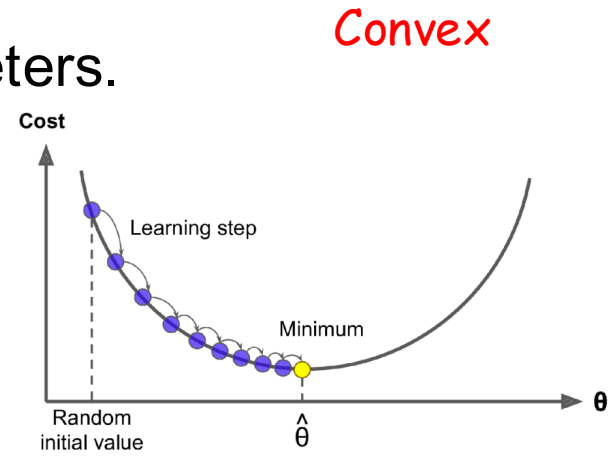
- Gradient Cost: Gradient is independent of ‘n’.
  - Each update uses one sample (or small mini-batch)
  - Each iteration cost is independent of dataset size n
  - One iteration is ~n times cheaper than GD
  - Produces noisy but unbiased gradient estimates

Stochastic Gradient Training

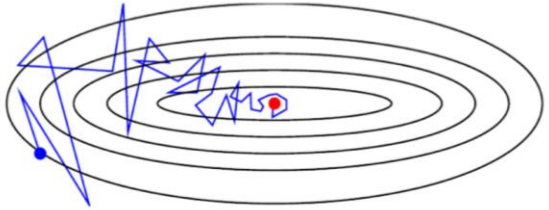
- We randomly samples gradients instead of averaging (all).
  - An average estimate --> faster to compute
- Why optimization is important?
  - NN objectives are highly non-convex (and worse with depth).
  - Optimization has huge influence on quality of model.



- Standard training method is SG:
  - Choose a random example ‘i’.
  - Use backpropagation to get gradient with respect to all parameters.
  - Take a small step in negative gradient direction.
    - Hard to get SG to even find a local minimum.



Stochastic Gradient (SG)



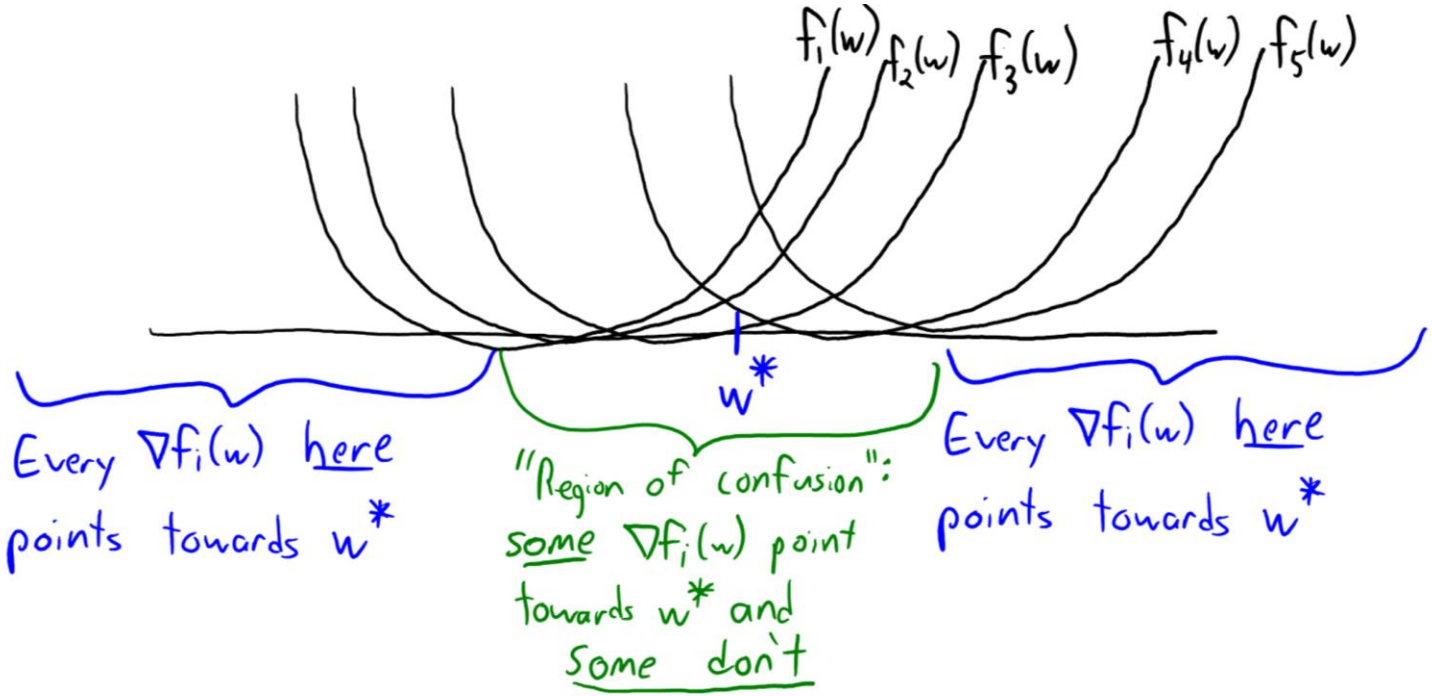
- Function $f_i(w)$ is error for example 'i'. $f(w) = \frac{1}{n}\sum_{i=1}^{n} f_i(w)$
- Iterations perform gradient descent on one random example 'i': $w^{t+1} = w^t - \alpha^t \nabla f_i(w^t)$
  - Cheap iterations even when 'n' is large, but doesn't always decrease 'f'.
  - Solves problem if α^t goes to 0 at an appropriate rate.

Property	Batch GD	SGD
Gradient accuracy	✅ Exact	⚠️ Noisy estimate
Convergence stability	✅ Smooth	⚠️ Oscillatory
Memory usage	✗ Entire dataset	✅ One sample
Speed on large data	✗ Slow	✅ Fast
Escaping local minima	✗ Prone to getting stuck	✅ Noise helps escape
Hyperparameter sensitivity	✅ Low	⚠️ Sensitive to learning rate

- SG converges very slowly:
  - Large dataset --> No other option.
- Improve performance by reducing variance use:
  - "mini-batches" or random samples but no one random sample.
  - "Variance-reduced" methods for finite training sets.

Variance

- "Confusion" is captured by a kind of variance of gradients
- Variance = 0: Every step goes in right direction. (outside of confusion region)
- Variance = Small: Just inside region of confusion.
- Variance = Large: Many steps will point in wrong direction.

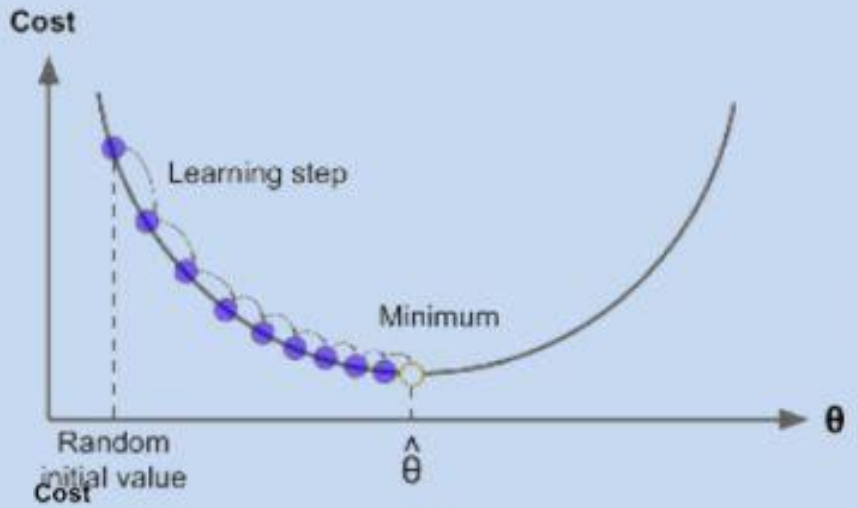


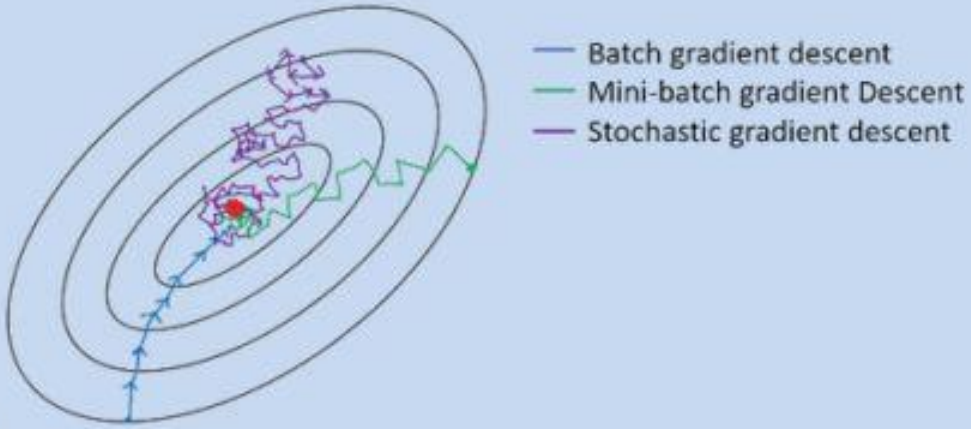
Batch and Mini-Batch GD

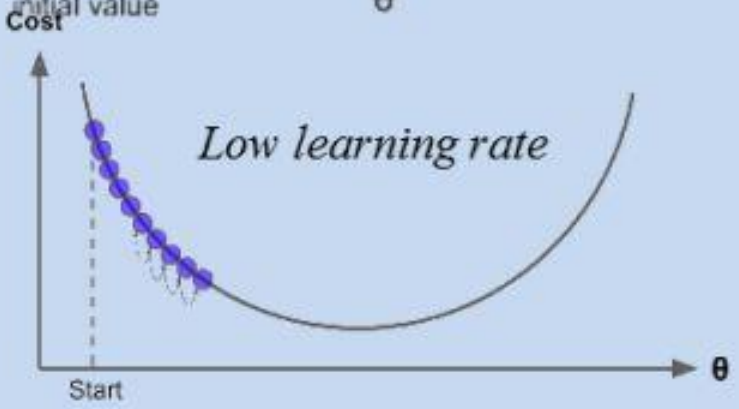
- At each GD step it calculates whole batch of training data.
  - SGD: single sample per update
  - SGD introduces noise but scales better
- Mini-batch GD computes gradients on small random sets of instances called mini-batches.
  - Faster than SGD (standard in DL)
- Batch GD stops at minimum (slow).
  - SGD and Mini-batch GD continue to walk around (faster).
  - All algorithms outputs similar models and predicts in exactly same way.

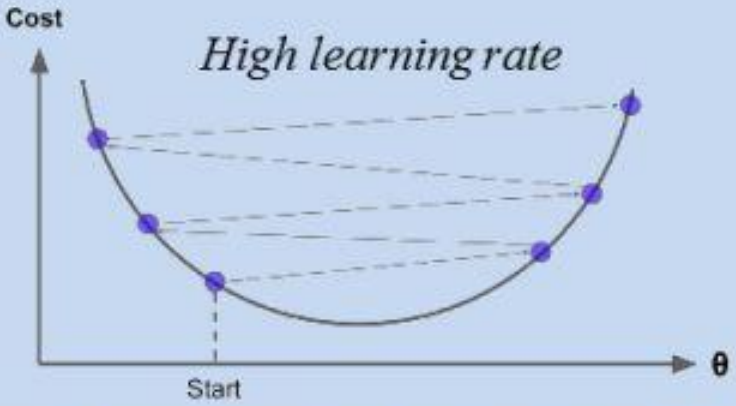
Stochastic Gradient Descent

is a powerful optimization algorithm used in machine learning to minimize the cost function of a model. It works by iteratively updating the parameters in small steps in the direction of the negative gradient of the cost function.









Source: Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow , By Aurélien Géron

Activation Functions:

A sufficiently large NN with nonlinear activation functions can approximate any continuous function (Theory)

Activation Functions

- It compute the output values of neurons in hidden layers in a neural network.
- It regulate the outputs of nodes and add a level of complexity that neural network without activation functions.
  - For linear functions the derivative is a constant and has no relation to the input.
  - Non-linear activation functions we can stack multiple hidden layers to create a more complex model.

Name	Definition
Linear	$g(x) = \alpha x$
Sigmoid	$g(x) = \frac{1}{1+e^{-x}} = sig(x)$
Tanh	$g(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 1 - \frac{2}{e^{2x}+1}$
Rectified Linear Unit (ReLU)	$g(x) = max(0, z)$
Leaky ReLU	$g(x) = \begin{cases} \alpha x, & \text{for } x < 0 \\ x, & \text{for } x \geq 0 \end{cases}$
Exponential Linear Unit (ELU)	$g(x) = \begin{cases} \alpha(e^x - 1), & \text{for } x < 0 \\ x, & \text{for } x \geq 0 \end{cases}$
Swish	$g(x) = \frac{x}{1+e^{-x}}$

Activation Functions

Name	Advantages	Disadvantages
Linear	• Activation values are real values; no binary activation	• No non-linearity can be learned • Derivative is constant - there is no relationship to an input x • With a constant gradient, the changes made by backpropagation is also constant
Sigmoid	• Smooth gradient, preventing 'jumps' in output values • Output values in range $[0, 1]$, normalizing the output of each neuron • Clear predictions: Low (High) values are very close to 0 (1)	• Vanishing gradient - for very high (low) values, there is almost no change to the prediction (known as vanishing gradient. This can lead to a slow learning or in the worst case there is no learning at all • Outputs are not zero centered making the optimization harder • Computationally expensive
Tanh	• Zero centered - modeling of inputs having strongly negative, neutral and positive values is facilitated	• Vanishing gradient - saturated activations kill the gradient
ReLU	• Learning procedure converges much faster than one with sigmoid/tanh activation functions • Computationally efficient - less mathematical operations • Avoids the vanishing gradient problem	• Dying ReLU Problem - when inputs approach zero or are negative, then the gradient is zero, hence, the network cannot perform backpropagation and there is no learning, leading to dead neurons • Range of ReLU is $[0, \infty)$ • Output is not zero-centered
Leaky ReLU	• Learning procedure converges much faster than one with sigmoid/tanh activation functions • Computationally efficient • Prevents the Dying ReLU Problem - a small positive slope in the negative domain enables backpropagation even for negative input values	• Results are not consistent being a consequence of the additional hyperparameter α regularizing the slope in the negative domain
ELU	• Advantage over ReLU, ELU can produce negative outputs,hence, no dead neurons • Closer to zero mean outputs	• Computation requires calling $exp(\cdot)$
Swish	• No dying ReLU problem • Has shown to have a slightly increase in accuracy over ReLU	• Slightly more expensive in terms of mathematical operations being necessary compared to ReLU

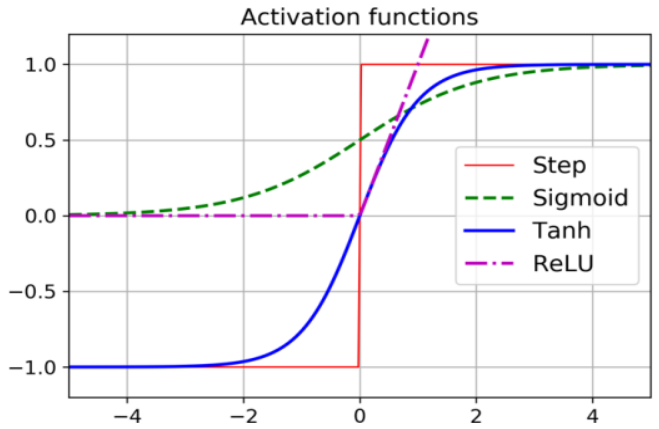
CMMI Tacit to Explicit

Activation Functions (AFs)

- How to choose?
  - Begin with ReLU and then move over to other if ReLU doesn't provide optimum results.
  - ReLU activation function should only be used in the hidden layers.
  - Sigmoid/Logistic and Tanh functions should not be used in hidden layers as they make the model more susceptible to problems during training (due to vanishing gradients).
  - AF used in hidden layers is chosen based on NN architecture type.
- Rule:
  - Regression: Linear AF
  - Binary Classification: Sigmoid/Logistic
  - Multiclass Classification: Softmax
  - Multilabel Classification: Sigmoid
  - CNN: ReLU
  - Recurrent Neural Network: Tanh and/or Sigmoid.

Activation Function

- defines how the weighted sum of inputs (plus bias) is transformed into the output of a neuron
  - Hidden layers use same activation function (not mandatory).
  - Output layer usually uses a different activation function.
  - Chosen based on the task:
    - Binary classification → Sigmoid
    - Multiclass (exclusive) → Softmax
    - Regression → Linear

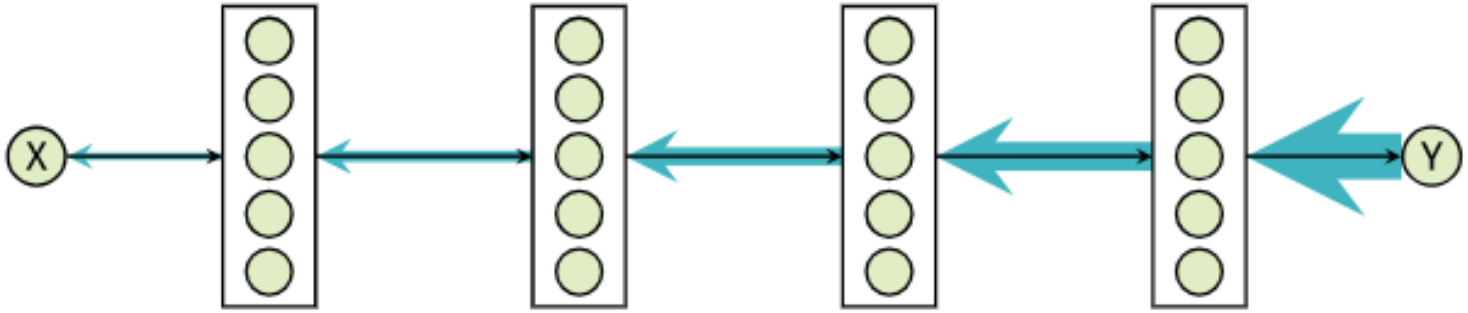


- Large DNN with nonlinear activations can theoretically approximate any continuous function.
  - Sigmoid (S-shaped) activation function was good.
  - ReLU works better in ANNs.
  - Softmax ensure all estimated probabilities between 0 and 1 and they add up to 1 (required for exclusive classes).

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow by Aurélien Géron, 2019.

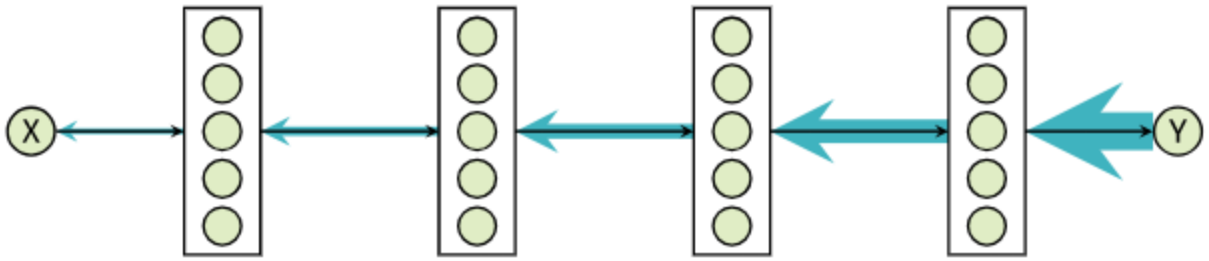
Vanishing Gradient Problem

- Gradients become extremely small as they are backpropagated through deep networks, causing negligible weight updates in early layers.
- Why:
  - Saturating activation functions (e.g., sigmoid, tanh)
  - Deep network depth
  - Bad weight initialization
- Consequences
  - Slow/stalled training
  - Model underfits
- Solutions
  - Proper weight initialization
  - Use non-saturating activations (ReLU, Leaky ReLU)
  - Batch Normalization



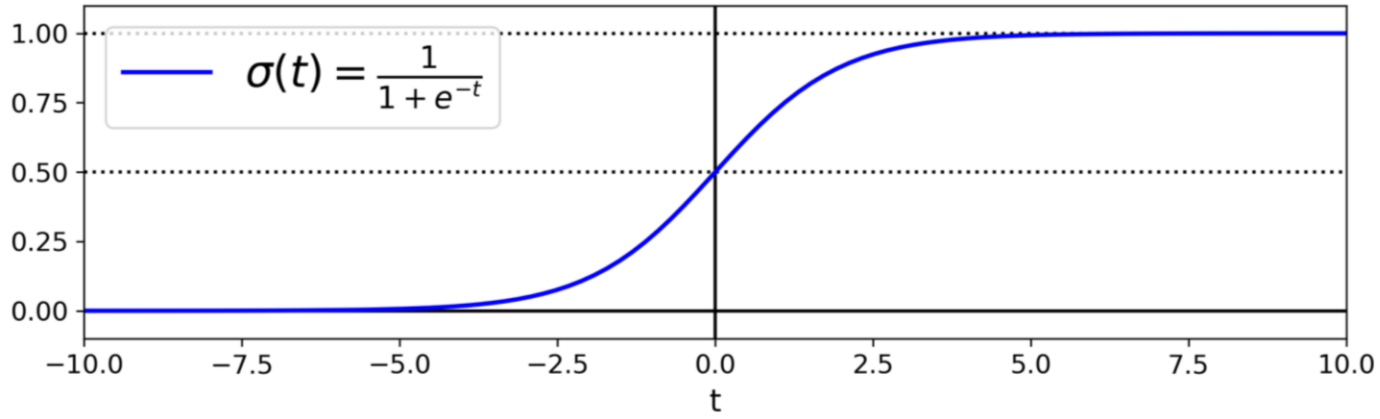
Exploding Gradient Problem

- Gradients grow excessively large during backpropagation due to repeated multiplication, leading to unstable updates.
- Why?
  - Deep networks
  - Large weight values
  - Improper initialization
- Consequences
  - Loss becomes unstable
  - Parameters diverge
- Solutions
  - Gradient clipping
  - Proper initialization
  - Batch Normalization
  - Careful learning rate tuning



Sigmoid Function

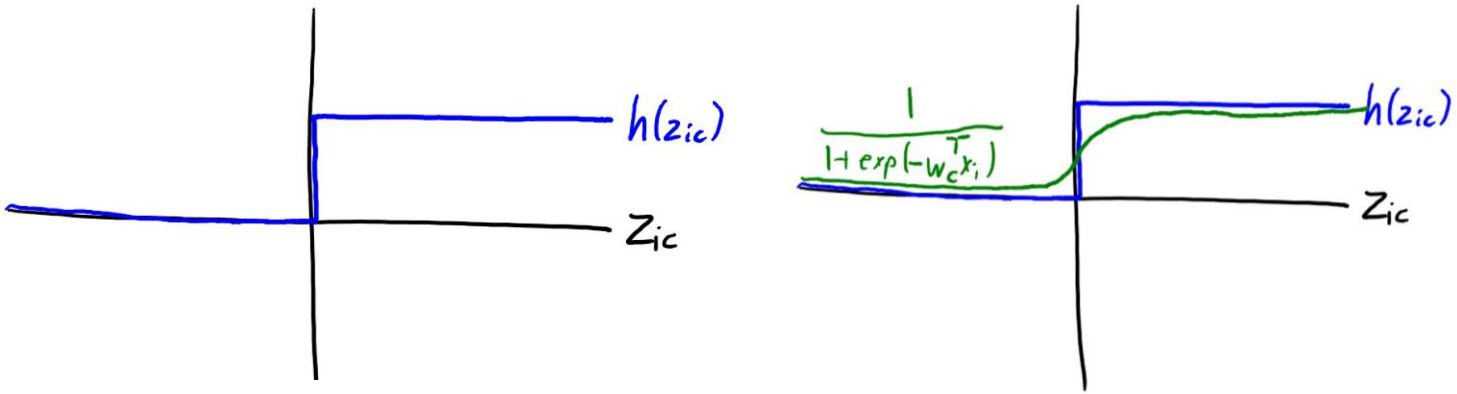
- Logistic (noted σ(·)) is a sigmoid function (i.e., S-shaped) that outputs between 0 and 1 (remember Estimating Probabilities by Logistic Regression).



- Sigmoid function is a type of activation function (or squashing function).
- Squashing functions limit output to a range between 0 and 1.
- Determines output behaviour of each node, or “neuron” in an ANN.
- Used in testing of ANNs.

Sigmoid

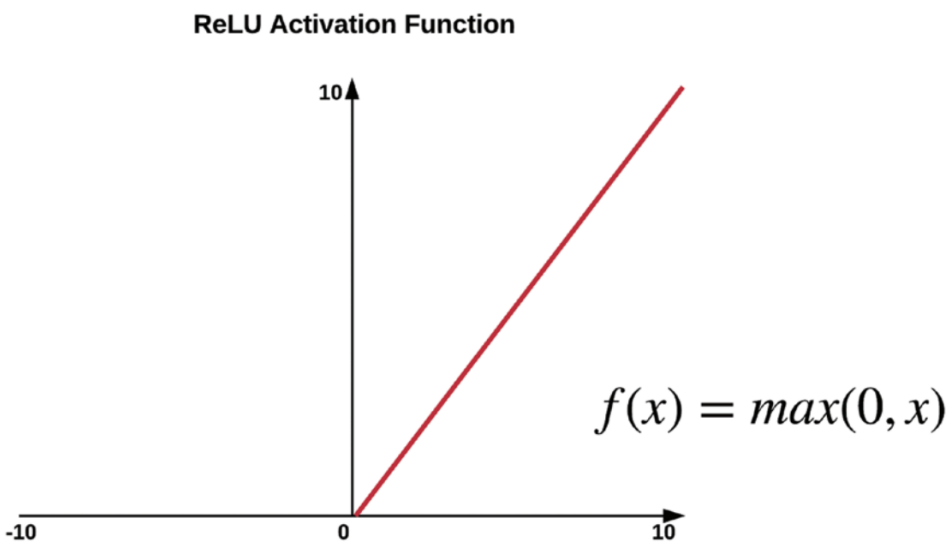
- Consider setting ‘h’ to define binary features z_i using:



- Each $h(z_i)$ can be viewed as binary feature.
- We can make 2^k objects by all the possible “part combinations”.
- Hard to optimize (non-differentiable/discontinuous).
- Sigmoid is a smooth approximation to these binary features.

Rectified Linear Units (ReLU)

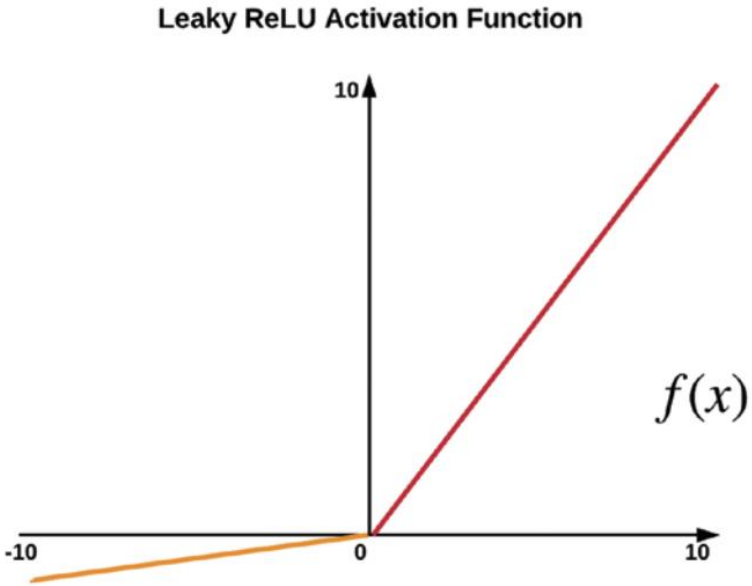
- ReLU mitigate the vanishing and exploding gradient problem.
- ReLU offers improvement on the tanh and sigmoid activation functions.
- ReLU works:
  - Set the activation to 0 for values x < 0
  - Set a linear slope of 1 when values x > 0



Bisong E. (2019) More on Optimization Techniques. In: Building Machine Learning and Deep Learning Models on Google Cloud Platform. Apress, Berkeley, CA

Leaky ReLU

- Some gradients can still die out during backpropagation with a large learning rate –-> use Leaky ReLU.
- Leaky ReLU works by setting the activation to a small negative slope when the value x < 0.



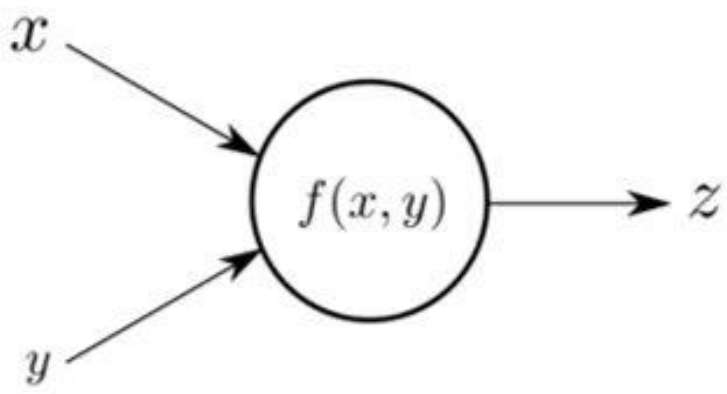
$$f(x) = \begin{cases} 0.01x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$$

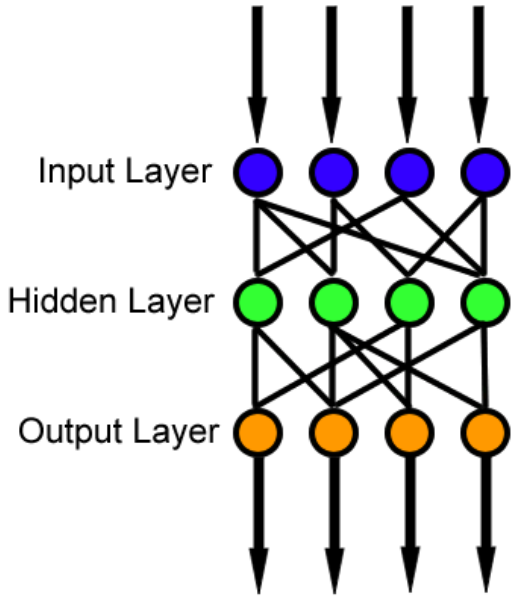
Bisong E. (2019) More on Optimization Techniques. In: Building Machine Learning and Deep Learning Models on Google Cloud Platform. Apress, Berkeley, CA

Feed-Forward Networks
& Backpropagation

Feed-Forward Networks (Prediction)

- There is no cyclic connection between network layers in Feed-forward networks.
- Input flows forward towards output via several hidden layers.
- Feedforward NN: The signal flows in one direction (inputs-->outputs).



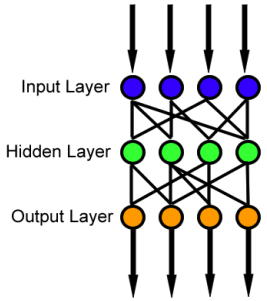


https://en.wikipedia.org/wiki/Feedforward_neural_network

Backpropagation

- Backpropagation computes the gradient of the neural network loss with respect to all parameters using the chain rule of calculus.
- Computing these gradients efficiently is called backpropagation.
  - For a dataset of size n:

$$f(v, W) = \frac{1}{2}\sum_{i=1}^{n}\left(v^T h(Wx_i) - y_i\right)^2$$ <--1-hidden-layer MLP with linear output layer



Where:

- x_i = input vector
- y_i = target
- W = hidden layer weights
- $h(\cdot)$ = hidden layer activation (applied element-wise)
- v = output layer weights
- $v^T h(Wx_i)$ = network prediction

- Hidden layer pre-activation:

$$z_i = Wx_i$$

- Hidden activation:

$$a_i = h(z_i)$$

- Output:

$$\hat{y}_i = v^T a_i$$

So the model is:

$$\hat{y}_i = v^T h(Wx_i)$$

- Linear model:

$$\hat{y} = w^T x$$

- 1 hidden layer:

$$\hat{y} = v^T h(Wx)$$

- 2 hidden layers:

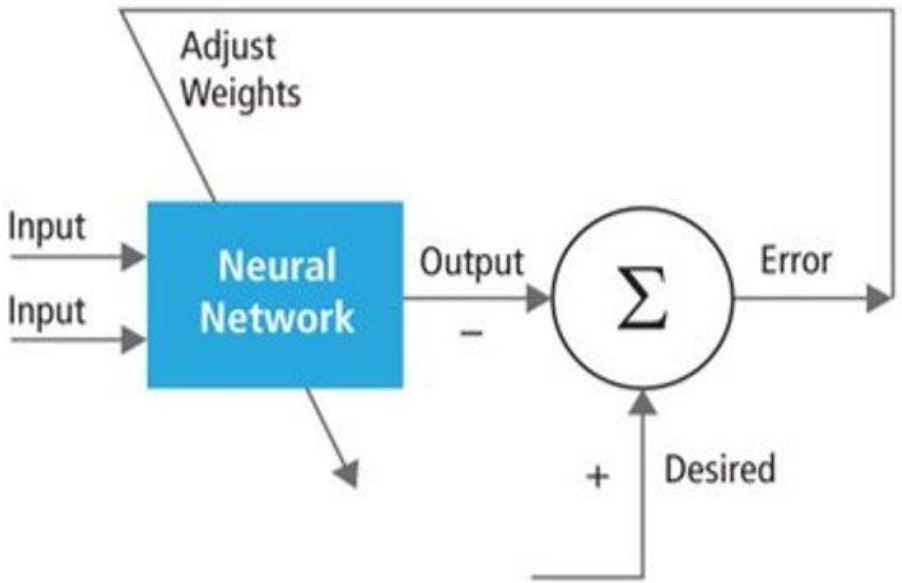
$$\hat{y} = v^T h(W^{[2]} h(W^{[1]} x))$$

- Deep network:

$$\hat{y} = W^{[L]} h(\ldots h(W^{[1]} x))$$

Backpropagation

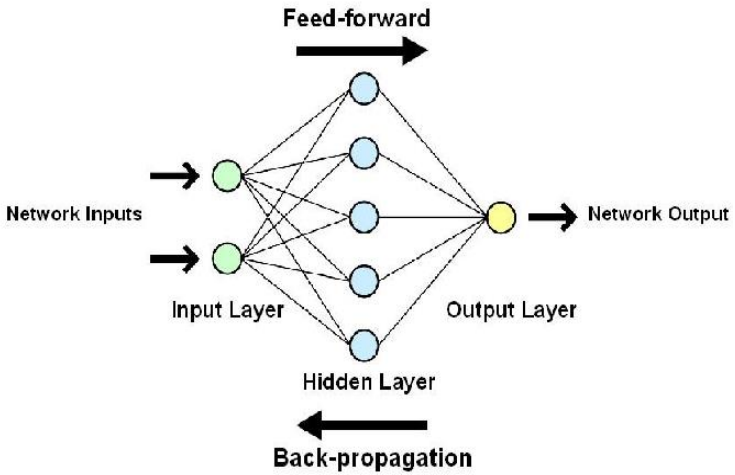
- Steps:
  - Forward pass: compute and cache activations
  - Compute loss (prediction error) at output
  - Compute output layer error gradient
  - Propagate error backward using W^T and chain rule
  - Compute weight/bias gradients from error signals
  - Update parameters: W



- Mainly:
  - Error propagates backward
  - It is scaled by output weights
  - Modulated by activation derivative
  - Multiplied by input

Backpropagation: Computational Cost

- Forward Pass Cost: Dominated by matrix multiplications: m = number of layers; Each hidden layer has k units, Input dimension = d
  - $\text{Cost}=O(dk+(m-1)k^2)$; First layer: $O(dk)$, Remaining layers: each $O(k^2)$;
  - For large m: $O(dk+mk^2)$
- Backward Pass Cost: Backward cost ≈ Forward cost
  - Same order of complexity as forward pass.
  - Each layer requires:
    - Matrix multiplication
    - Element-wise activation derivatives
    - For large m: $O(dk+mk^2)$



Backpropagation

- Uses chain rule to compute gradients efficiently
  - Forward pass → compute predictions
  - Backward pass → propagate errors and update weights
  - Cost comparable for forward and backward passes

- Prevent overfitting:
  - L2 regularization
  - Dropout
  - Early stopping
- Stop training when validation error increases

Optimization Techniques

Optimization Techniques

- Babysitting or ‘Trial and Error’ method is manual tuning.
  - Problems due to large number of hyper-parameters, complex models, time-consuming model evaluations, and non-linear hyper-parameter interactions.

- Grid search (GS) most commonly-used methods
  - An exhaustive search or a brute-force method
  - Works by evaluating the Cartesian product of user-specified finite set of values
  - Problem: Inefficiency for high-dimensionality hyper-parameter configuration space
    - Since number of evaluations increases exponentially to number of hyper-parameters growth

Yang, Li, and Abdallah Shami. "On hyperparameter optimization of machine learning algorithms: Theory and practice." *Neurocomputing* 415 (2020): 295-316.

Optimization Techniques

- Random search (RS): like GS.
  - Randomly selects a pre-defined number of samples between upper and lower bounds as candidate hyper-parameter values, and then trains these candidates until defined budget is exhausted.
  - With a limited budget, RS can explore a larger search space than GS.
  - Problem: unnecessary function evaluations since it does not exploit previously well-performing regions.

- Gradient-based optimization: traditional technique.
  - After randomly selecting a data point, it moves towards opposite direction of largest gradient to locate next data point.
    - Local optimum can be reached after convergence.
  - Gradient-based algorithms have a fast convergence speed to reach local optimum.
  - Can use to optimize learning rate in NN.

Yang, Li, and Abdallah Shami. "On hyperparameter optimization of machine learning algorithms: Theory and practice." *Neurocomputing* 415 (2020): 295-316.

Optimization Techniques

- Gradient-based optimization: Grid Search is used to optimize the parameters of a model (e.g. C, kernel and gamma for SVC) using an internal Cross-Validation
- Approach
  - Grid: Specify a set of possible values for each hyperparameter you want to tune.
  - Train and Evaluate: Grid search systematically tries every possible combination of hyperparameter values from the grid.
  - Cross-Validation: For each combination, it evaluates the model's performance using cross-validation on the training data.
  - Select the Best: The combination of hyperparameters that yields the highest average cross-validation score.

Optimization Techniques

- Grid search is exhaustive.
- If you have many hyperparameters or large ranges of values for each, making the search computationally expensive and time-consuming.
- Selecting appropriate ranges and values for the grid requires some domain knowledge or experimentation.
- Alternatives: For very large search spaces RandomizedSearchCV (samples a fixed number of combinations randomly)

```python
# Define the estimator
svm_model = SVC()

# Define the grid of hyperparameters to search
param_grid = {
    'C': [0.1, 1, 10, 100],            # Regularization parameter
    'gamma': [1, 0.1, 0.01, 0.001],    # Kernel coefficient for 'rbf'
    'kernel': ['rbf', 'linear']        # Type of kernel
}
```

```python
# Instantiate GridSearchCV
# cv=5 means 5-fold cross-validation
# scoring='accuracy' specifies the metric to optimize
grid_search = GridSearchCV(estimator=svm_model,
                           param_grid=param_grid,
                           cv=5,
                           scoring='accuracy',
                           verbose=1, # Optional: prints progress
                           n_jobs=-1) # Optional: use all available CPU cores
```

Adam optimization algorithm

- An extension to SGD for computer vision and NLP.
- Derived from adaptive moment estimation.
- In Adam, a separate learning rate is maintained for each parameter weight (adapted and updated individually).
  - Selects a smaller learning rate for frequently updated parameters and
  - a larger one for parameters corresponding to rare features.
- Adam realizes the benefits of both Adaptive Gradient Algorithm (AdaGrad) and Root Mean Square Propagation (RMSProp).

https://arxiv.org/abs/1412.6980

Adam optimization algorithm

- Adam Configuration Parameters:
  - alpha (learning rate): Proportion that weights are updated (e.g. 0.001). Larger values (e.g. 0.3) and Smaller values (e.g. 1.0E-5)
  - Beta1 and Beta2: Control the exponential smoothing of previous gradients and their squares, respectively.
    - Beta1: for smoothing gradients
    - Beta2 handles the smoothing of gradient squares.
  - Epsilon: Is a very small number to prevent any division by zero in the implementation (e.g. 10E-8).
- Example: TensorFlow: learning_rate=0.001, beta1=0.9, beta2=0.999, epsilon=1e-08.

https://arxiv.org/abs/1412.6980

AdamW

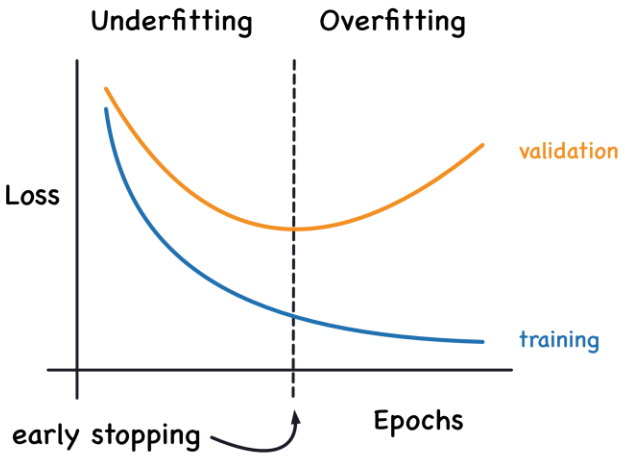
- AdamW is an optimization algorithm in deep learning (2024)
- A modification to the Adam optimizer
  - It decouple weight decay from gradient-based updates.
  - Decoupling was introduced to address overfitting arise when using standard Adam (for large-scale neural network models).
- Adam’s strengths: adaptive learning rates and computational efficiency.
- AdamW to achieve superior convergence for BERT and GPT.
- AdamW can yield improved validation accuracy, faster convergence, and better generalization compared to both standard Adam and SGD with momentum (for large-scale applications).
- Application: NLP, computer vision, reinforcement learning, and generative modeling

Early Stopping

Early Stopping

- Regularization is crucial to neural network performance:
  - L2-regularization, early stopping, dropout.

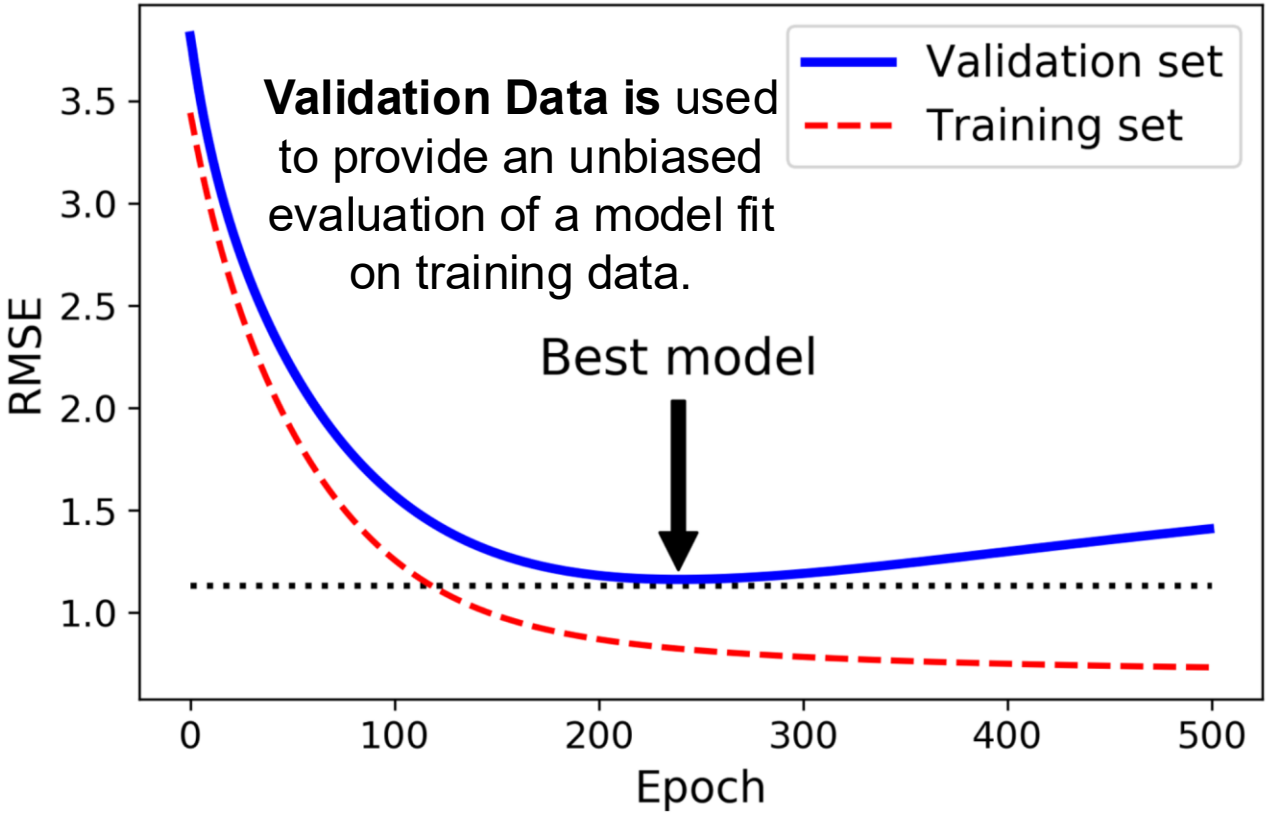
- Second common type of regularization is “early stopping”:
  - Monitor the validation error as we run Stochastic Gradient.
  - Stop algorithm if validation error starts increasing (overfitting data*).
  - Stop training as soon as validation error reaches minimum.



- Overfitting --> Model performs well on training data, but it does not generalize well.
- Underfitting --> Model is too simple to learn underlying structure of data.

Example

- A high-degree Polynomial Regression model being trained with GD.



- As epochs go by algorithm learns
- Its prediction error (RMSE) on training set goes down
- Prediction error on validation set is also going down.

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow by Aurélien Géron, 2019.

Summary

- Gradient Descent (GD): Iterative optimization to minimize objective
- **Convergence**:
  - Convex → global minimum
  - Non-convex → stationary point (local minima/saddle)
- Requires proper learning rate (α) and feature scaling
- **Learning Rate**
  - Controls update magnitude and convergence speed
  - Too small → slow convergence
  - Too large → oscillation/divergence
  - Adaptive optimizers reduce manual tuning
- **Stochastic & Mini-Batch GD**
  - Batch GD: full dataset per step (stable, slow)
  - SGD: one sample per update (fast, noisy)
  - Mini-batch GD: standard in deep learning
  - Variance in gradients affects convergence behavior

Summary

- **Advanced Optimizers**
  - Adam: adaptive per-parameter learning rates
  - Combines momentum + RMSProp
  - AdamW: decoupled weight decay → better generalization
- **Hyperparameter Optimization**
  - Grid Search: exhaustive, expensive in high dimensions
  - Random Search: better exploration with limited budget
  - Gradient-based tuning possible for some parameters
- **Common Activations**
  - Sigmoid: outputs (0,1), prone to vanishing gradients
  - Tanh: zero-centered but still vanishing issue
  - ReLU: mitigates vanishing gradients, computationally efficient
  - Leaky ReLU: avoids dying ReLU problem